

Computational Statistics Exercises 1

Zhi Zhou

2026-09-06

```
# Load  
library(tidyverse)
```

Exercise 1 - Variance after a change of origin

Task 1, 2 and 3

```
# Reproduce the exact dataset from the exercise  
set.seed(1)  
z <- rnorm(100000, mean = 0, sd = 3)  
offsets <- c(0, 1e4, 1e8, 1e12, 1e14)  
n <- length(z)  
  
# Define the two-pass and one-pass algorithms  
# two-pass algorithm  
var2pass <- function(x) {  
  m <- mean(x)  
  sum((x - m) ** 2) / (n - 1)  
}  
  
# one-pass computational formula  
var1pass <- function(x) {  
  (sum(x ** 2) - sum(x) ** 2 / n) / (n - 1)  
}  
  
# Run all three methods at every offset  
res <- data.frame(  
  a = offsets,  
  var_r = NA,  
  two_pass = NA,  
  one_pass = NA  
)  
variances <- sapply(offsets, function(o) {  
  x <- o + z # Task 1 !!!  
  c( # Task 2 !!!  
    var_r = var(x),  
    two_pass = var2pass(x),  
    one_pass = var1pass(x)  
  )  
}) # Task 3 !!!
```

```

res$var_r <- variances["var_r", ]
res$two_pass <- variances["two_pass", ]
res$one_pass <- variances["one_pass", ]
res

##          a      var_r two_pass      one_pass
## 1 0e+00 9.063530 9.063530 9.063530e+00
## 2 1e+04 9.063530 9.063530 9.063530e+00
## 3 1e+08 9.063530 9.063530 7.864399e+00
## 4 1e+12 9.063529 9.063529 0.000000e+00
## 5 1e+14 9.063590 9.063590 -1.441166e+12

```

Task 4

```

# Compare with var(z)
rel <- data.frame(
  a      = offsets,
  var_r  = abs(res$var_r - var(z)) / var(z),
  two_pass = abs(res$two_pass - var(z)) / var(z),
  one_pass = abs(res$one_pass - var(z)) / var(z)
)
# Print the comparison table
rel

##          a      var_r      two_pass      one_pass
## 1 0e+00 0.000000e+00 0.000000e+00 0.000000e+00
## 2 1e+04 1.959895e-16 1.959895e-16 1.079250e-09
## 3 1e+08 1.395445e-13 1.397405e-13 1.323029e-01
## 4 1e+12 8.833966e-08 8.833966e-08 1.000000e+00
## 5 1e+14 6.572327e-06 6.572327e-06 1.590072e+11

# Print the reference value
cat("var(z) =", var(z), "\n")

## var(z) = 9.06353

# Find the threshold, where first offset with more than 10% one-pass error
cat("one-pass error > 10% first at a =", res$a[which(rel$one_pass > 0.1)[1]], "\n")

## one-pass error > 10% first at a = 1e+08

# Figure: relative error vs offset (log-log)
# Reshape the table to long format, one row per (offset, method)
rel_long <- data.frame(
  a = rep(offsets, 3),
  method = rep(c("var(x)", "two-pass", "one-pass"), each = length(offsets)),
  rel = c(rel$var_r, rel$two_pass, rel$one_pass)
)
# Drop a=0, as error is exactly 0 and no log
rel_long <- rel_long |> filter(a > 0)
# Floor tiny values so log works
rel_long$rel <- rel_long$rel |> (\(x) pmax(x, 1e-16))()

ggplot(rel_long, aes(
  x = a,
  y = rel,

```

```

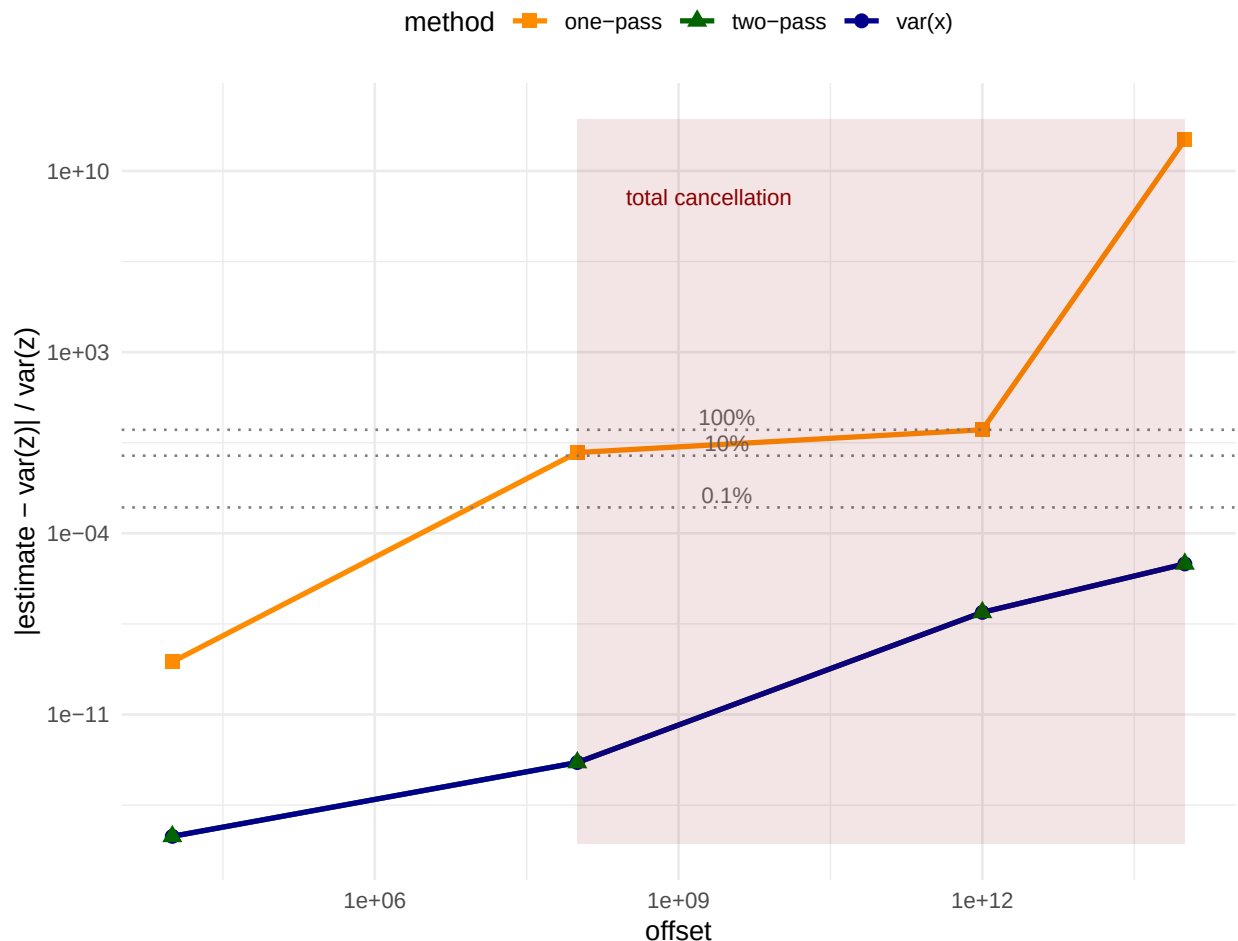
color = method,
shape = method
)) +
geom_line(linewidth = 1) +
geom_point(size = 2.5) +
scale_x_log10() +
scale_y_log10() +
coord_cartesian(ylim = c(1e-16, 1e12)) +
geom_hline(
  yintercept = c(1e-3, 1e-1, 1),
  linetype = "dotted",
  color = "grey50"
) +
annotate(
  "text",
  x = 3e9,
  y = c(3e-3, 3e-1, 3),
  label = c("0.1%", "10%", "100%"),
  color = "grey40",
  size = 3
) +
annotate(
  "rect",
  xmin = 1e8,
  xmax = 1e14,
  ymin = 1e-16,
  ymax = 1e12,
  fill = "darkred",
  alpha = 0.1
) +
annotate(
  "text",
  x = 2e9,
  y = 1e9,
  color = "darkred",
  label = "total cancellation",
  size = 3
) +
scale_color_manual(values = c(
  "var(x)" = "darkblue",
  "two-pass" = "darkgreen",
  "one-pass" = "darkorange"
)) +
scale_shape_manual(values = c(
  "var(x)" = 16,
  "two-pass" = 17,
  "one-pass" = 15
)) +
labs(
  x = "offset",
  y = "|estimate - var(z)| / var(z)",
  title = "Task 4. Relative erroer of the three estimates vs var(Z)",
  subtitle = "set.seed(1), n = 1e5, sd = 3"

```

```
) +
theme_minimal() +
theme(legend.position = "top")
```

Task 4. Relative error of the three estimates vs var(Z)

set.seed(1), n = 1e5, sd = 3



`var(x)` and the two-pass method stay accurate no matter how large the offset is. The quick one-pass formula only works well up to an offset of about 10^4 . At offset 10^8 , it is already off by roughly 13% (7.864 instead of 9.063530), and with larger offsets it fails completely. The reason is that it subtracts two huge, nearly equal numbers, $\sum x^2$ and $\sum x/n$, which makes rounding error blow up.

Task 5

The math is simple: add a constant to everything, and the variance does not change because the mean shifts with it, so the distances from the mean stay exactly the same.

The trouble only starts when a computer uses the shortcut formula. That formula has to deal with enormous squared numbers, and when numbers get that huge, the computer starts rounding them off. That rounding error often ends up being bigger than the actual variance we are trying to find. Then, when we subtract those massive rounded sums to get a small result, the rounding error takes over completely. We can even get negative variance, which obviously makes no sense.

The smarter method finds the average first, subtracts it from every point, and then squares the leftovers.

Since the leftovers are small, the rounding stays small too, so the result actually means something. That is why the built-in functions use this safer approach.

Task 6

Yes, we should definitely preprocess. If the numbers are huge, like around a trillion, but the differences we actually care about are really tiny, the computer will struggle to get the math right. The big numbers just get in the way.

The fix is simple. Pick a reference number, maybe the average or some known starting point, and subtract it from every measurement. We are not losing any useful information because things like variance and standard deviation do not change when shift everything by the same point.

Now the numbers are small and manageable. We can actually see the real variation instead of it getting lost in all those zeros. It also saves the computer from making silly rounding errors, so our results will actually be correct.

Exercise 2 - A probability that disappears

Task 1

```
# The x values given in the exercise
x <- c(5, 8, 10, 20, 30, 40)

# Task 1: Compute the upper-tail probability in the three ways
m1 <- 1 - pnorm(x)
m2 <- pnorm(x, lower.tail = FALSE)
m3 <- pnorm(x, lower.tail = FALSE, log.p = TRUE)

cbind(x, "1 - pnorm" = m1, "lower.tail=FALSE" = m2, "log.p = TRUE" = m3)

##      x      1 - pnorm lower.tail=FALSE log.p = TRUE
## [1,]  5 2.866516e-07      2.866516e-07 -15.06500
## [2,]  8 6.661338e-16      6.220961e-16 -35.01344
## [3,] 10 0.000000e+00      7.619853e-24 -53.23129
## [4,] 20 0.000000e+00      2.753624e-89 -203.91716
## [5,] 30 0.000000e+00      4.906714e-198 -454.32124
## [6,] 40 0.000000e+00      0.000000e+00 -804.60844
```

Task 2

The formula `1 - pnorm(x)` is already inaccurate at $x = 8$, with a relative error of about 6.6%. From $x = 10$ onward, it gives exactly 0. This happens because once the true upper-tail probability is smaller than about 1.1×10^{-16} , `pnorm(x)` rounds to exactly 1 in the computer. Then `1 - 1` returns 0, and the tiny tail probability is lost.

Task 3

This is a problem called cancellation. For large x , `pnorm(x)` is extremely close to 1. Computers store only about 16 significant digits, so numbers very near 1 have gaps of about 2.2×10^{-16} between them. The tail probability `1 - pnorm(x)` is the tiny difference between two nearly equal numbers. When we subtract them, the computer loses the small digits that held the actual answer. Once `pnorm(x)` rounds to exactly 1, the result becomes exactly 0. The function `pnorm(x)` itself is computed correctly. The error happens only in the subtraction step, not inside `pnorm()`.

Task 4

```
cbind(x, "log.p" = m3, "exp(log.p)" = exp(m3))
```

```
##      x      log.p      exp(log.p)
## [1,]  5 -15.06500  2.866516e-07
## [2,]  8 -35.01344  6.220961e-16
## [3,] 10 -53.23129  7.619853e-24
## [4,] 20 -203.91716  2.753624e-89
## [5,] 30 -454.32124  4.906714e-198
## [6,] 40 -804.60844  0.000000e+00
```

Basically, we can convert a log p-value back to a regular number up to a point, say Z around 30, where the result is still super tiny but the computer can handle it. Push to $Z = 40$, though, and the number gets so ridiculously small that it is below the computer's floor. It is just rounds to zero.

The log p-value itself stays perfectly fine no matter how extreme we get. The computer can keep track of it easily. The only thing that fails is the conversion back. So if we just stick with log space, we are buying roughly 450 extra orders of magnitude to work with, which is more than enough for just about any real situation.

Task 5

When we have an extreme statistics like $Z = 9$, the p-value is absurdly small, about 10^{-19} . We do not want to work with that number directly because it is easy for the computer to mess up. For instance, if we try to calculate it as 1 minus something, we will get killed by rounding errors, and if we try to store it as a plain number, it might just round down to zero.

The smart move is to work in log space. Instead of the p-value itself, carry around its log. Compare things by checking if the log p-value is smaller than the log of our threshold. It is the exact same test mathematically, but it will not blow on us. And if we need to combine p-values from multiple tests, we just add their logs instead of multiplying the tiny numbers, which would underflow to zero.

We only exponentiate back at the very end if we absolutely need the actual p-value, and even then, it is fine. This is pretty standard stuff in genomics. People routinely store and plot things as negative log p-values because it is just way more practical.

Exercise 3 - Comparing two models without comparing their likelihoods

```
# reproduce the data in the exercise
set.seed(2)
y <- rbinom(5000, size = 1, prob = 0.03)

p1 <- 0.03
p2 <- 0.035
```

Task 1

```
L1 <- prod(ifelse(y == 1, p1, 1 - p1))
L2 <- prod(ifelse(y == 1, p2, 1 - p2))
c(L1, L2)
```

```
## [1] 0 0
```

Task 2

```
l1 <- sum(y * log(p1) + (1 - y) * log(1 - p1))
l2 <- sum(y * log(p2) + (1 - y) * log(1 - p2))
c(l1, l2)
```

```
## [1] -750.1850 -748.6221
```

Task 3

If we compute the likelihood directly, both come out to zero. The numbers are just too small for the computer to handle, so they both round down to the same thing. That makes it look like the two models are equally good, which is completely misleading.

But if we use log-likelihood instead, the difference is clear. The log for p2 is -748.62, which is higher than -750.19 for p1. So p2 is actually preferred. It is almost five times more likely than p1.

The data was actually generated from $p1 = 0.03$. But for this particular sample, the observed success rate happened to be closer to p2. So the likelihood is doing its job correctly by favoring p2. The directly likelihoods would incorrectly told us the models are tied, when in reality they are not even close. The log scale saves us from that false conclusion.

Task 4

```
l1 - l2
```

```
## [1] -1.562957
```

This is more useful than the likelihoods themselves because it gives you the log-likelihood ratio, a table number that does not get too big or too small for the computer, no matter how many observations we have. When we are comparing models, what really matters is the relative support between them, not the absolute likelihood value. On the top of that, these log ratios add up nicely across independent observations, and twice their difference is exactly the chi-square statistic we would use for a standard likelihood ratio test.

Task 5

```
LR12 <- exp(l1 - l2)
LR21 <- exp(l2 - l1)
c(LR12, LR21)
```

```
## [1] 0.2095156 4.7729139
```

Task 6

When you multiply a bunch of small probabilities together, the result gets so tiny that computer often just round it to zero. That is why people switch to logs. logs turn those tiny products into normal-sized sums and keep everything stable.

In Bayesian computation, for example, the posterior is proportional to the likelihood times the prior, and the Bayes factors are ratios of two very small integrals. If you try to compute those directly with raw likelihoods, they underflow. So people work with log-posteriors and log-marginal-likelihoods, which keep the numbers in a reasonable range.

The same idea applies to the maximum likelihood estimation. Maximizing the likelihood is the same as maximizing the log-likelihood, because the log function is monotone. It does not change where the maximum is. But the log version turns a product of many likelihood terms into a sum, avoiding underflow. And all the standard tools for optimization and standard errors, like Fisher information, are built on log-likelihoods.

Finally, in sequential analysis, things like Wald tests or the sequential probability ratio test, you keep multiplying likelihood ratios over time. Those products underflow fast. But the log-likelihood ratio is just a running sum. That is much easier to track and compare against thresholds. The idea of “comparing models without computing their likelihoods” means working with the accumulated log ratio instead of the raw product, which would otherwise blow up or vanish.

Exercise 4 - Diagnosing a numerically suspicious covariance matrix

```
# reproduce the data
set.seed(3)
n <- 5000
u <- rnorm(n)
x1 <- 1e8 + u
x2 <- 1e8 + u + rnorm(n, sd = 1e-4)
X <- cbind(x1, x2)
```

Task 1

```
cov(X)

##           x1           x2
## x1 1.012977 1.012978
## x2 1.012978 1.012979
```

Task 2

```
cor(x1, x2)

## [1] 1
```

Task 3

```
eigen(cov(X))$values

## [1] 2.025956e+00 4.992558e-09
```

Task 4

```
onepasscov <- function(a, b) {
  n <- length(a)
  (sum(a * b) - sum(a) * sum(b) / n) / (n - 1)
}

cov1pass <- matrix(
  c(
    onepasscov(x1, x1),
    onepasscov(x1, x2),
    onepasscov(x2, x1),
    onepasscov(x2, x2)
  ),
  nrow = 2,
  byrow = T
```

```
)
cov1pass

##           [,1]      [,2]
## [1,] 3.277455 1.638728
## [2,] 1.638728 0.000000
```

Task 5

```
cov(X) - cov1pass

##           x1      x2
## x1 -2.2644781 -0.6257497
## x2 -0.6257497  1.0129787
```

Task 6

```
Xc <- scale(X, center = T, scale = F)
cov1pass_c <- matrix(
  c(
    onepasscov(Xc[, 1], Xc[, 1]),
    onepasscov(Xc[, 1], Xc[, 2]),
    onepasscov(Xc[, 2], Xc[, 1]),
    onepasscov(Xc[, 2], Xc[, 2])
  ),
  nrow = 2,
  byrow = T
)
cov1pass_c
```

```
##           [,1]      [,2]
## [1,] 1.012977 1.012978
## [2,] 1.012978 1.012979
all.equal(cov1pass_c, cov(X))
```

```
## [1] "Attributes: < Length mismatch: comparison on first 1 components >"
```

Task 7

Warning signs. First, the correlation is almost perfect. Second, one of the eigenvalues is tiny, around 5×10^{-9} , while the other is much larger. The condition number works out to about 10^8 , which means any error in the matrix gets blown up by a factor of 100 million when we invert it. And sure enough, the inverse itself is full of huge entries, like $\pm 10^8$, which is a red flag. Third, the whole thing is incredibly twitchy. If we change a single entry in the covariance matrix by just a tiny amount, the inverse can shift. That is no great, especially since any rounding error in computing the covariance itself will get magnified by that same factor.

Safe approaches. We have got a few options. One is to regularize the matrix by adding a small constant to the diagonal before inverting. That is the ridge approach, and it is common fix. Another is to use a pseudo inverse or an SVD, which lets us ignore or tone down the problematic direction instead of making it worse.

Exercise 5 - Whitening is not unique

Task 1

```
set.seed(4)
n <- 1000
p <- 5
Z <- matrix(rnorm(n * p), n, p)
St <- matrix(0.3, p, p)
diag(St) <- 1
U <- chol(St)
X <- Z %%% U
X <- scale(X, center = T, scale = F)
S <- cov(X)
S

##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 0.9395492 0.3062005 0.2684128 0.3435962 0.3439137
## [2,] 0.3062005 1.0049620 0.3414014 0.3544817 0.3836703
## [3,] 0.2684128 0.3414014 0.9643780 0.3028375 0.3036733
## [4,] 0.3435962 0.3544817 0.3028375 0.9981628 0.3161630
## [5,] 0.3439137 0.3836703 0.3036733 0.3161630 1.0502244

eigen(S)$values

## [1] 2.3023101 0.7143548 0.7080609 0.6390129 0.5935378
```

Task 2

```
eig <- eigen(S)
Q <- eig$vectors
Lambda <- eig$values
W_PCA <- diag(1 / sqrt(Lambda)) %%% t(Q)
Z_PCA <- X %%% t(W_PCA)
cov(Z_PCA)

##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 1.000000e+00 -9.420832e-17 2.320016e-16 -1.882110e-16 -4.861283e-16
## [2,] -9.420832e-17 1.000000e+00 2.048503e-16 6.567415e-17 1.683715e-17
## [3,] 2.320016e-16 2.048503e-16 1.000000e+00 -2.214831e-16 -1.106271e-16
## [4,] -1.882110e-16 6.567415e-17 -2.214831e-16 1.000000e+00 -1.639944e-16
## [5,] -4.861283e-16 1.683715e-17 -1.106271e-16 -1.639944e-16 1.000000e+00

max(abs(cov(Z_PCA) - diag(p)))

## [1] 4.861283e-16
```

Task 3

```
W_ZCA <- Q %%% diag(1 / sqrt(Lambda)) %%% t(Q)
Z_ZCA <- X %%% t(W_ZCA)
cov(Z_ZCA)

##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 1.000000e+00 -3.806801e-16 -6.310914e-17 5.741833e-17 2.933255e-16
## [2,] -3.806801e-16 1.000000e+00 -1.947039e-16 3.400859e-16 1.433235e-17
```

```
## [3,] -6.310914e-17 -1.947039e-16 1.000000e+00 5.376270e-16 2.024031e-17
## [4,] 5.741833e-17 3.400859e-16 5.376270e-16 1.000000e+00 2.929845e-16
## [5,] 2.933255e-16 1.433235e-17 2.024031e-17 2.929845e-16 1.000000e+00

max(abs(cov(Z_ZCA) - diag(p)))

## [1] 8.881784e-16
```

Task 4

```
L <- t(chol(S))
W_chol <- solve(L)
Z_chol <- X %*% t(W_chol)
cov(Z_chol)

##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 1.000000e+00 -4.029862e-17 6.795099e-17 6.942085e-17 -3.824800e-17
## [2,] -4.029862e-17 1.000000e+00 5.652546e-17 4.099218e-17 -2.188328e-17
## [3,] 6.795099e-17 5.652546e-17 1.000000e+00 1.209265e-17 -2.698658e-17
## [4,] 6.942085e-17 4.099218e-17 1.209265e-17 1.000000e+00 3.188092e-17
## [5,] -3.824800e-17 -2.188328e-17 -2.698658e-17 3.188092e-17 1.000000e+00

max(abs(cov(Z_chol) - diag(p)))

## [1] 1.110223e-16
```

Is L a matrix square root of S ? No. A square root A satisfies $A^2 = S$. The Cholesky factor satisfies $LL^T = S$, but L is lower triangular and not symmetric, so $L^2 \neq S$. The symmetric square root is $\sqrt{S} = Q\sqrt{\Lambda}Q^T$, which does satisfy $(\sqrt{S})^2 = S$, L is a square root factor, not a square root.

Task 5

```
max(abs(Z_PCA - Z_ZCA))

## [1] 5.843427

max(abs(Z_PCA - Z_chol))

## [1] 6.020725

# all have identity covariance
c(max(abs(cov(Z_PCA) - diag(p))), max(abs(cov(Z_ZCA) - diag(p))), max(abs(cov(Z_chol) - diag(p))))

## [1] 4.861283e-16 8.881784e-16 1.110223e-16

# they differ by orthogonal rotations
R <- solve(crossprod(Z_PCA), crossprod(Z_PCA, Z_ZCA))
max(abs(R %*% t(R) - diag(p)))

## [1] 3.996803e-15
```

Task 6

Whitening is not a one-of-a-kind transformation. Suppose we have got a matrix W that whitens our data. If we take that result and rotate it using any orthogonal matrix R , which just means we are spinning the data around without stretching it. The data stay whitened, because rotating an identity covariance matrix keeps it as an identity.

Since there are infinite ways to rotate, whitening does not give you a unique result. The common ones we see: PCA, ZCA, and Cholesky. They are just a few standard ways to pick that rotation, but they are definitely not the only ones out there.

Task 7

```
X2 <- X
X2[, 5] <- X2[, 1] + 1e-7 * rnorm(n)
X2 <- scale(X2, center = T, scale = F)
S2 <- cov(X2)
eigen(S2)$values

## [1] 2.527930e+00 1.014136e+00 6.722842e-01 6.322515e-01 9.325873e-15
eigen(S2)$values[5]/eigen(S2)$values[1]

## [1] 3.689134e-15
eig2 <- eigen(S2)
W2_PCA <- diag(1 / sqrt(eig2$values)) %*% t(eig2$vectors)
Z2_PCA <- X2 %*% t(W2_PCA)
max(abs(cov(Z2_PCA) - diag(p)))

## [1] 0.4382963
max(abs(W2_PCA))

## [1] 7322177
L2 <- t(chol(S2)); W2_Chol <- solve(L2)
Z2_Chol <- X2 %*% t(W2_Chol)
max(abs(cov(Z2_Chol) - diag(p)))

## [1] 0.005247107
max(abs(W2_Chol))

## [1] 9795419
```

The problem is that tiny eigenvalue makes you divide by a very small number, which magnifies all the little errors and noise in our data. So instead of getting a clean, nice result, the whitening gets messed up. The whole thing becomes twitchy, the tiniest change in the input can completely change the output. And Cholesky will not help either because it is not built to handle a matrix that is nearly singular.

Task 8

When a matrix is almost singular, one of its eigenvalues is nearly zero. Standard whitening divides by that tiny eigenvalue, and everything blows up. So we need a workaround.

A simple fix is to add a small number to the diagonal before whitening. That raises the tiny eigenvalue just enough to make division safe. Another option is to keep only the biggest eigenvalues and ignore the rest. Then the whitened data lives in a smaller space. Or we can use the pseudo-inverse, which does something similar.

On the computing side, these fixes stop us from dividing by very small numbers, so we avoid huge weights and amplified noise. The covariance matrix then looks close to an identity matrix, which is the goal.

On the statistics side, we are making a trade-off. We throw away some directions, which is fine if they were mostly noise. But we lose the original meaning of the variable scales. Regularization also shrinks those weak directions to zero, adding a little bias. So we trade some interpretability for much better numerical stability.